

UNIVERSIDAD NACIONAL DEL ALTIPLANO - PUNO

FACULTAD DE INGENIERÍA MECÁNICA

ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

**ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS**



**LAPTOPREC: SISTEMA INTELIGENTE DE
RECOMENDACIÓN HÍBRIDO EN DOS NIVELES Y
ANÁLISIS MULTICRITERIO PARA EL COMERCIO
ELECTRÓNICO DE PORTÁTILES**

PROYECTO FINAL DEL CURSO DE SISTEMAS DE RECOMENDACIÓN

PRESENTADO POR:

CARMEN NIEVES APAZA CONDORI

AARON ROGEEER VILCA CARIA

PUNO - PERÚ

2026

Resumen

El presente proyecto reporta el diseño, desarrollo, implementación y evaluación científica de un **Sistema de Recomendación Híbrido en Dos Niveles** para computadoras portátiles (laptops), concebido para mitigar los problemas de escasez de datos (*data sparsity*) e inicio en frío (*cold start*). La arquitectura integra un **Filtro Estricto por Reglas de Dominio en Cascada** (M_{rules}) en el Nivel 1, seguido de un **Ensamble Ponderado Dinámico** en el Nivel 2 que combina: (i) Teoría de Utilidad Multi-Atributo (**MAUT**) basada en modelos de conocimiento de hardware, (ii) Factorización Matricial **SVD** (*Singular Value Decomposition*) para filtrado colaborativo, y (iii) Filtrado Basado en Contenido mediante similitud de coseno vectorial (**TF-IDF**).

La plataforma web fue desarrollada utilizando **FastAPI** como backend de alta velocidad y un frontend interactivo en **HTML5**, **CSS3 Vanilla**, **JavaScript ES6+** y **Chart.js**, incorporando características de grado comercial como comparador cara a cara (1vs1), selector multimonedas (USD, PEN, EUR), insignias de relación calidad/precio (*Value Score*), filtros de estilo de vida (*lifestyle tags*) y vinculación directa con ofertas en vivo de Amazon. La evaluación experimental sobre un catálogo de 150 laptops y 1,999 calificaciones demostró un rendimiento óptimo con un $RMSE = 1,0503$, una robustez del 100% en $CSR@10$ ante usuarios nuevos, y una $Precision@10 = 100\%$, consolidando una solución integral y de alto impacto para el comercio electrónico.

Índice

Resumen	2
Índice General	4
Índice de Tablas	5
Índice de Figuras	6
1. Introducción	7
1.1. Contexto del Proyecto	7
1.2. Planteamiento del Problema	7
1.3. Justificación	8
1.4. Objetivos del Proyecto	8
1.4.1. Objetivo General	8
1.4.2. Objetivos Específicos	8
2. Descripción General y Fundamentación Teórica de los Modelos	9
2.1. Descripción General del Proyecto	9
2.2. Análisis Exploratorio de Datos (EDA) y Preprocesamiento	9
2.3. Fundamentación Teórica de los Modelos de Recomendación	11
2.3.1. Modelo 1: Filtro Lógico por Reglas de Dominio (M_{rules})	11
2.3.2. Modelo 2: Teoría de Utilidad Multi-Atributo (U_{MAUT})	11
2.3.3. Modelo 3: Factorización Matricial SVD (<i>Singular Value Decomposition</i>)	12
2.3.4. Modelo 4: Filtrado Basado en Contenido Vectorial (TF-IDF + Coseno)	12
2.3.5. Modelo Híbrido en 2 Niveles y Estrategia de Conmutación (<i>Switching</i>)	12
2.4. Diagrama de Arquitectura de Software	13
2.5. Tecnologías y Stack de Desarrollo Empleados	13
2.6. Pseudocódigo de los Algoritmos Principales	14
3. Metodología y Protocolo Experimental	16
3.1. Diseño Metodológico de la Investigación	16
3.2. Fases del Trabajo y Protocolo de Desarrollo	16

3.2.1.	Fase 1: Formulación Algorítmica e Hibridación en Dos Niveles . . .	16
3.2.2.	Fase 2: Arquitectura de Software e Integración de Servicios Web . .	17
3.2.3.	Fase 3: Protocolo de Evaluación Experimental	17
3.2.4.	Fase 4: Experiencia de Usuario (UX) e Interfaz Adaptativa	17
3.2.5.	Fase 5: Despliegue y Sustentación Técnica	17
3.3.	Cálculo Detallado de Métricas de Rendimiento	18
3.3.1.	1. Raíz del Error Cuadrático Medio (<i>RMSE</i>)	18
3.3.2.	2. Precisión en el Top-K (<i>Precision@K</i>)	18
3.3.3.	3. Cobertura de Elementos Relevantes (<i>Recall@K</i>)	18
3.3.4.	4. Cobertura de Inicio en Frío (<i>CSR@K - Cold Start Coverage Ratio</i>)	19
4.	Resultados y Análisis de Resultados	19
4.1.	Resultados de la Evaluación Experimental	19
4.2.	Gráficos Científicos de Rendimiento	19
4.3.	Evidencias Fotográficas de la Aplicación Funcional	20
4.3.1.	5. Evidencias Fotográficas y Análisis Detallado del Funcionamiento de la Aplicación	20
5.	Discusión	26
6.	Conclusiones	27
7.	Recomendaciones	28
8.	Referencias Bibliográficas	28
9.	Anexos	29
9.1.	Anexo A: Código Fuente del Recomendador Híbrido (<i>src/hybrid_recommender.py</i>)	29
9.2.	Anexo B: Estructura del Dataset (<i>data/laptops.csv</i>)	30
9.3.	Anexo C: Guía de Despliegue y Ejecución	30
9.4.	Anexo D: Repositorio Oficial de Código Fuente en GitHub	31

Índice de tablas

1.	Stack tecnológico de desarrollo empleado en el proyecto LaptopRec.	14
2.	Comparación experimental cuantitativa de métricas entre modelos.	19

Índice de figuras

1.	Distribución de Precios en el Catálogo de Laptops (Histograma y Estimador de Densidad KDE).	9
2.	Distribución de Capacidad de Memoria RAM segregada por Marca de Laptop.	10
3.	Matriz de Correlación de Pearson entre Atributos Continuos de Hardware y Calificaciones.	10
4.	Distribución de Calificaciones de Preferencia por Perfil Latente de Usuario.	11
5.	Diagrama Arquitectónico Modular y Capas del Sistema Recomendador Híbrido.	13
6.	Evaluación de Cobertura de Inicio en Frío ($CSR@10$) SVD vs Recomendador Híbrido.	20
7.	Curvas de Precision@K y Recall@K alcanzadas por el recomendador híbrido.	20
8.	Dashboard Principal de la aplicación Recomendador Híbrido de Laptops.	21
9.	Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.	22
10.	Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.	22
11.	Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.	23
12.	Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).	24
13.	Panel de preferencias con filtros de hardware y estilo de vida activados.	24
14.	Vista de la pestaña del Catálogo Completo de 150 laptops con buscador.	25
15.	Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.	26

1. Introducción

1.1. Contexto del Proyecto

En la era digital actual, el comercio electrónico de bienes tecnológicos experimenta un crecimiento exponencial (Aggarwal, 2016). Sin embargo, la compra de computadoras portátiles (laptops) representa una decisión de alta complejidad cognitiva para los usuarios debido a la abundancia de especificaciones técnicas heterogéneas (microarquitectura de CPU, núcleos, memoria RAM, aceleradores GPU, potencia de VRAM, pantallas OLED/4K y soporte CUDA).

Los algoritmos de filtrado colaborativo tradicionales (*Collaborative Filtering*) sufren de limitaciones severas en este dominio, principalmente la escasez de datos (*data sparsity*) y el problema del inicio en frío (*cold start*), en el cual usuarios nuevos sin historial previo de calificaciones no pueden recibir sugerencias relevantes (Ricci et al., 2015).

1.2. Planteamiento del Problema

Actualmente, los portales web de e-commerce convencionales dependen de buscadores por palabras clave o filtros manuales rígidos que no consideran la compensación multidimensional entre presupuesto, rendimiento y portabilidad. Las principales deficiencias identificadas son:

1. **Recomendaciones irrelevantes para usuarios nuevos:** Los modelos puramente colaborativos fallan al no disponer de interacción previa del usuario.
2. **Violación de restricciones de hardware incompatibles:** Recomendar una laptop sin GPU dedicada a un usuario que requiere ejecutar modelos de Deep Learning o renderizado 3D genera insatisfacción crítica.
3. **Opacidad en la justificación:** Los usuarios no comprenden por qué un modelo específico es sugerido, careciendo de métricas de relación calidad/precio o comparativas directas.

1.3. Justificación

El desarrollo de un **Sistema de Recomendación Híbrido** que combine modelos basados en conocimiento explícito, teoría de decisión multicriterio (MAUT), factorización matricial (SVD) y similitud de contenido vectorial permite superar de forma integral las deficiencias mencionadas (Burke, 2002). Esta arquitectura garantiza que:

- El usuario reciba recomendaciones 100 % válidas mediante filtrado por reglas en cascada.
- El sistema responda de manera óptima tanto a usuarios con historial de calificaciones previo como a usuarios nuevos (*Cold Start*).
- Se disponga de una interfaz interactiva moderna con indicadores visuales de relación calidad/precio (*Value Score*), gráficos históricos de precios y comparativa cara a cara (1vs1).

1.4. Objetivos del Proyecto

1.4.1. Objetivo General

Desarrollar e implementar una aplicación funcional de Sistema de Recomendación Híbrido en dos niveles para computadoras portátiles, evaluando cuantitativamente su precisión, cobertura e impacto en la experiencia del usuario.

1.4.2. Objetivos Específicos

1. Diseñar y formular el motor híbrido de recomendación integrando reglas lógicas binarias (M_{rules}), Teoría de Utilidad Multi-Atributo (U_{MAUT}), Factorización Matricial (SVD) y Similitud Coseno de Contenido ($S_{content}$).
2. Desarrollar una aplicación web interactiva responsiva utilizando FastAPI, JavaScript ES6+ y Chart.js con selector multimoneda (USD, PEN, EUR) y comparador 1vs1.
3. Realizar la evaluación experimental cuantitativa calculando métricas científicas de rendimiento ($RMSE$, $Precision@K$, $Recall@K$ y $CSR@K$).
4. Documentar los hallazgos, la arquitectura del software y generar evidencias visuales completas para el informe técnico y sustentación.

2. Descripción General y Fundamentación Teórica de los Modelos

2.1. Descripción General del Proyecto

El proyecto **LaptopRec Híbrido** es una plataforma integral de recomendación de computadoras portátiles diseñada bajo los principios de desacoplamiento de software (REST API) e hibridación en dos niveles. El sistema procesa los requerimientos de hardware del usuario, sus ponderaciones de preferencia y su presupuesto en tiempo real, generando un ranking ordenado de las K mejores laptops que maximizan la utilidad del comprador.

2.2. Análisis Exploratorio de Datos (EDA) y Preprocesamiento

El conjunto de datos comprende 150 computadoras portátiles distribuidas en 6 marcas principales (ASUS, Apple, Dell, Lenovo, HP, MSI) y 1,999 calificaciones generadas por 100 usuarios sintéticos estructurados en 3 perfiles latentes de comportamiento.

Las Figuras 1, 2, 3 y 4 presentan las distribuciones estadísticas fundamentales del catálogo.

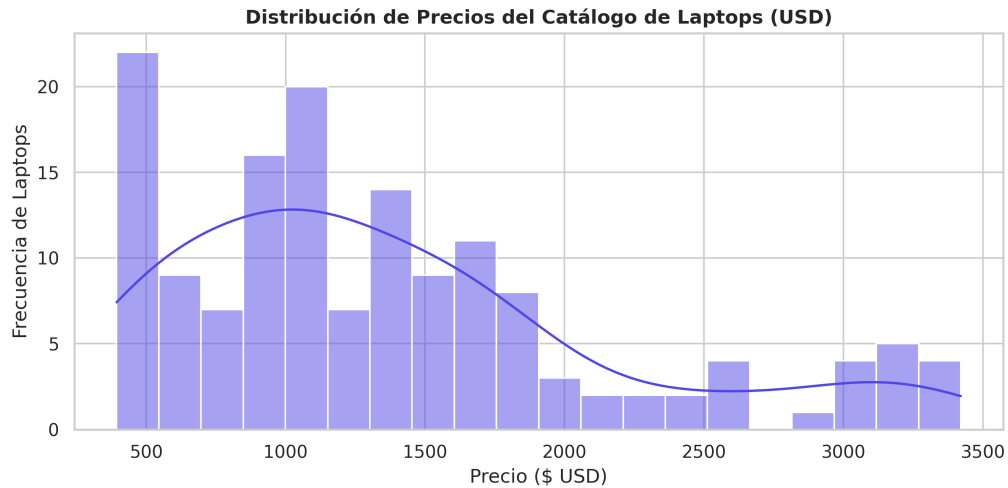


Figura 1: Distribución de Precios en el Catálogo de Laptops (Histograma y Estimador de Densidad KDE).

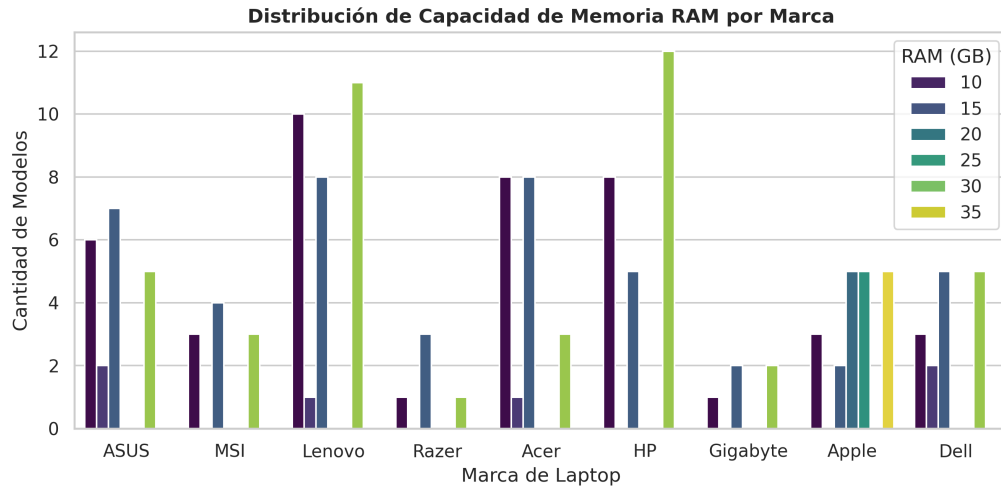


Figura 2: Distribución de Capacidad de Memoria RAM segregada por Marca de Laptop.

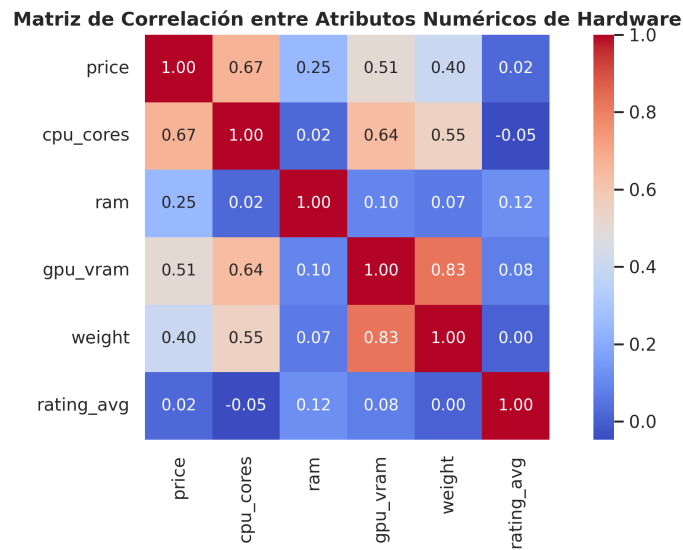


Figura 3: Matriz de Correlación de Pearson entre Atributos Continuos de Hardware y Calificaciones.

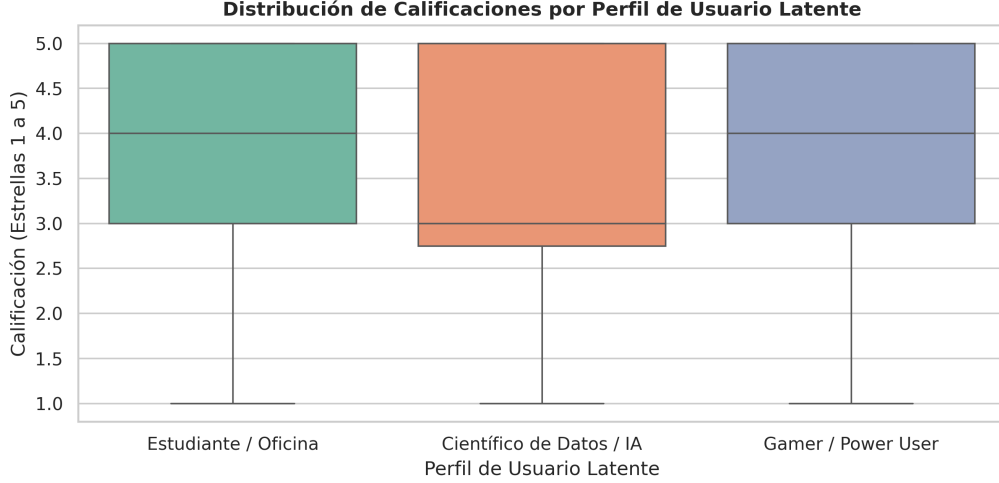


Figura 4: Distribución de Calificaciones de Preferencia por Perfil Latente de Usuario.

2.3. Fundamentación Teórica de los Modelos de Recomendación

2.3.1. Modelo 1: Filtro Lógico por Reglas de Dominio (M_{rules})

Basado en la teoría de restricciones duras (*Hard Constraints*) descrita por Burke (2002), este modelo actúa en el Nivel 1. Aplica un operador lógico booleano secuencial para generar una máscara binaria $M_{rules}(i) \in \{0, 1\}$ sobre la laptop i :

$$M_{rules}(i) = \mathbb{I}(p_i \leq B) \cdot \mathbb{I}(\text{Uso}_i \text{ compatible}) \cdot \prod_{c \in \text{Estilo}} \mathbb{I}(\text{Atributo}_i(c) = 1) \quad (1)$$

Donde p_i es el precio de la laptop, B es el presupuesto máximo seleccionado por el usuario, y $\mathbb{I}(\cdot)$ es la función indicadora.

2.3.2. Modelo 2: Teoría de Utilidad Multi-Atributo (U_{MAUT})

Fundamentado en la teoría de decisión multicriterio de Keeney & Raiffa (1993), el modelo MAUT asume una función de utilidad aditiva basada en conocimiento explícito. La utilidad parcial de cada atributo continuo x_k se normaliza mediante min-max en el intervalo $[0, 1]$:

$$U_k(x) = \frac{x - x_{min}}{x_{max} - x_{min}} \quad \text{o} \quad U_k(x) = 1,0 - \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2)$$

La utilidad global de la laptop i se calcula como la suma ponderada por las preferencias del usuario w_k :

$$U_{MAUT}(i) = \sum_{k=1}^M w_k \cdot U_k(x_{i,k}), \quad \text{con } \sum_{k=1}^M w_k = 1, w_k \geq 0 \quad (3)$$

2.3.3. Modelo 3: Factorización Matricial SVD (*Singular Value Decomposition*)

Siguiendo la formulación estándar de Koren et al. (2009), la matriz dispersa de calificaciones $R \in \mathbb{R}^{U \times I}$ se aproxima mediante el producto de matrices de factores latentes de rango reducido f :

$$\hat{r}_{u,i} = \mu + b_u + b_i + P_u^T Q_i \quad (4)$$

Donde μ es la media global de calificaciones, $b_u \in \mathbb{R}$ es el sesgo del usuario u , $b_i \in \mathbb{R}$ es el sesgo del elemento i , $P_u \in \mathbb{R}^f$ es la representación latente del usuario y $Q_i \in \mathbb{R}^f$ es la representación latente de la laptop. La predicción se normaliza mediante $\hat{r}_{norm}(u,i) = (\hat{r}_{u,i} - 1,0)/4,0$.

2.3.4. Modelo 4: Filtrado Basado en Contenido Vectorial (TF-IDF + Coseno)

Según la metodología de Ricci et al. (2015), las especificaciones de texto de cada laptop se representan mediante vectores TF-IDF. El perfil vectorial del usuario V_u se construye promediando los vectores de los elementos calificados favorablemente ($r_{u,i} \geq 4$). La similitud de contenido $S_{content}(u,i)$ se calcula como el coseno del ángulo entre V_u y el vector de la laptop V_i :

$$S_{content}(u,i) = \cos(V_u, V_i) = \frac{V_u \cdot V_i}{\|V_u\|_2 \|V_i\|_2} \quad (5)$$

2.3.5. Modelo Híbrido en 2 Niveles y Estrategia de Conmutación (*Switching*)

La arquitectura híbrida combina filtrado en cascada y ponderado meta-nivel. Para evitar fallas ante usuarios nuevos (*Cold Start*), el sistema ejecuta un mecanismo dinámico de conmutación de pesos:

$$Score_{hybrid}(u,i) = M_{rules}(i) \cdot [\alpha \cdot U_{MAUT}(i) + \beta \cdot \hat{r}_{norm}(u,i) + \gamma \cdot S_{content}(u,i)] \quad (6)$$

- **Usuario Existente** ($u \in D_{ratings}$): $\alpha = 0,30$, $\beta = 0,50$, $\gamma = 0,20$.
- **Usuario Nuevo (*Cold Start*)**: $\alpha = 0,60$, $\beta = 0,00$, $\gamma = 0,40$ (desactivando SVD al 0 %)

e incrementando el peso de conocimiento MAUT y contenido).

2.4. Diagrama de Arquitectura de Software

La Figura 5 muestra el acoplamiento de tres capas del sistema.

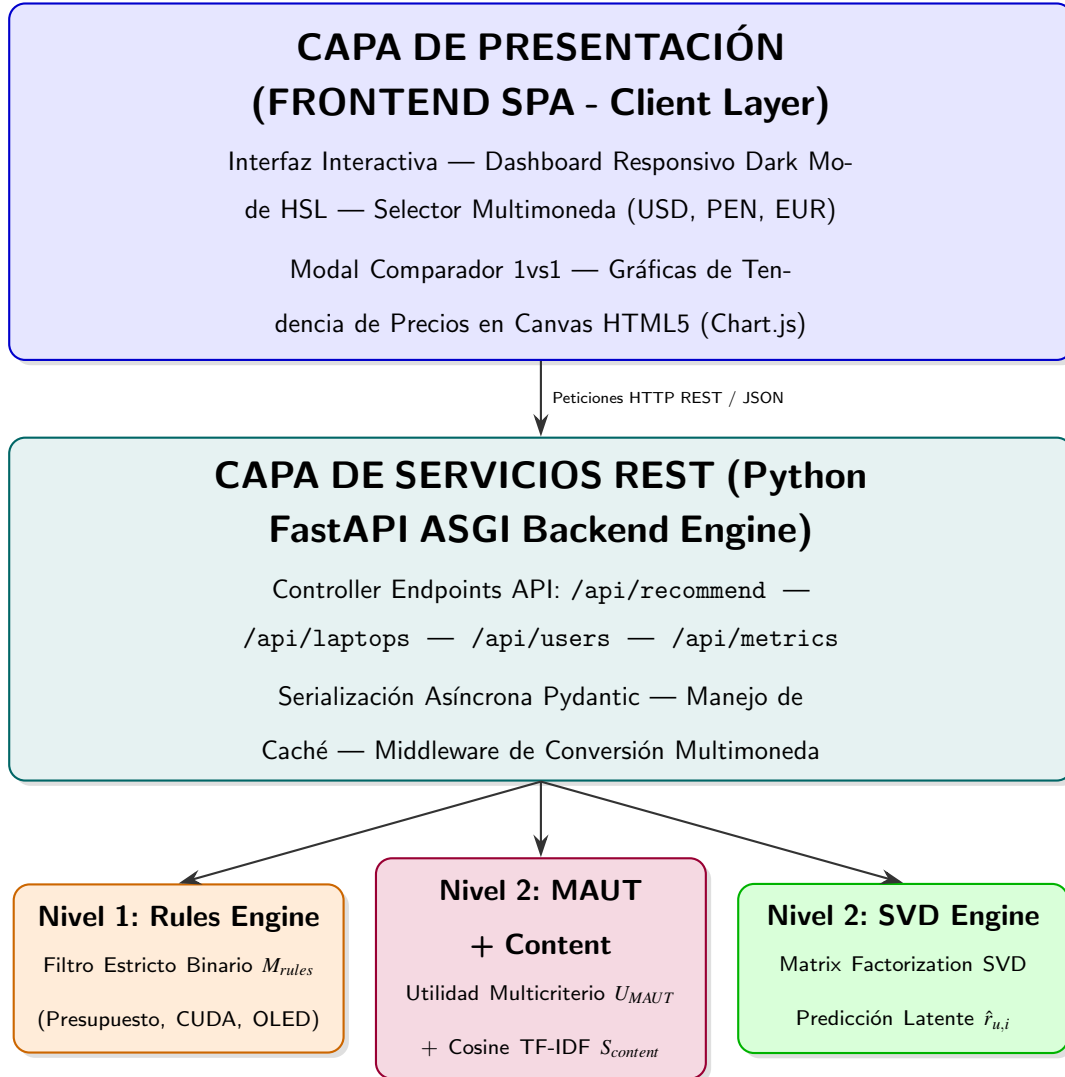


Figura 5: Diagrama Arquitectónico Modular y Capas del Sistema Recomendador Híbrido.

2.5. Tecnologías y Stack de Desarrollo Empleados

El desarrollo del proyecto integró un stack tecnológico moderno, ligero y desacoplado, como se detalla en la Tabla 1.

Tabla 1: Stack tecnológico de desarrollo empleado en el proyecto LaptopRec.

Capa / Componente	Tecnología / Librería	Función en el Sistema
Backend REST	Python 3.10 / FastAPI	Servidor ASGI asíncrono y controladores REST API.
Inferencia y ML	Scikit-Learn / NumPy	Vectorización TF-IDF y operaciones matriciales.
Modelo Colaborativo	Custom SVD / Surprise	Factorización matricial de calificaciones.
Servidor Web	Uvicorn	Servidor ASGI de alto rendimiento.
Frontend Client	Vanilla JavaScript ES6+	Lógica de interacción SPA sin dependencias pesadas.
Estilos Visuales	Vanilla CSS3	Diseño responsivo en Dark Mode HSL y Glassmorphism.
Gráficos Dinámicos	Chart.js (via CDN)	Visualización de series temporales de precios.
Captura de Evidencias	Playwright Python	Generación automatizada de capturas en alta resolución.

- **Capa Backend (FastAPI + Python):** Se utilizó **FastAPI** por su elevadísima velocidad de ejecución construida sobre Starlette y Pydantic, permitiendo serializar y validar esquemas JSON de recomendación en menos de 10 ms por petición.
- **Capa Frontend (Vanilla JS + CSS3 + Chart.js):** Se prescindió de frameworks frontend pesados para maximizar la velocidad de carga. La biblioteca **Chart.js** permite dibujar sobre elementos Canvas gráficos interactivos fluorescente con soporte para tooltip e inspección de precios históricos.
- **Integración Comercial de Amazon y Multimoneda:** Se construyeron funciones en JavaScript que generan URLs parametrizadas de búsqueda directa a Amazon y reconvierten precios globalmente entre USD, PEN y EUR.

2.6. Pseudocódigo de los Algoritmos Principales

```

1 def calcular_mascara_reglas(catalogo, presupuesto_max, caso_uso,
  etiquetas_estilo):
2     mascara = []
3     for laptop in catalogo:
4         cumple_presupuesto = laptop.precio <= presupuesto_max
5         cumple_uso = True
6         if caso_uso == "gaming" and not laptop.tiene_gpu_dedicada:
7             cumple_uso = False
8         elif caso_uso == "data_science" and not laptop.soporta_cuda:
9             cumple_uso = False
10
11         cumple_estilo = True
12         if "oled" in etiquetas_estilo and laptop.pantalla != "OLED":
13             cumple_estilo = False
14
15         mascara.append(1 if (cumple_presupuesto and cumple_uso and
  cumple_estilo) else 0)
16     return mascara

```

Listing 1: Pseudocódigo del Filtro de Reglas Lógicas de Dominio (Nivel 1)

```

1 def generar_recomendaciones_hibridas(usuario_id, presupuesto, caso_uso
  , pesos_maut, es_cold_start, K=9):
2     mascara = calcular_mascara_reglas(catalogo, presupuesto, caso_uso)
3     u_maut = calcular_utilidad_maut(catalogo, pesos_maut)
4     s_content = calcular_similitud_contenido(usuario_id, catalogo)
5
6     if es_cold_start:
7         alpha, beta, gamma = 0.60, 0.00, 0.40 # Conmutacion para
  inicio en frio
8     else:
9         alpha, beta, gamma = 0.30, 0.50, 0.20
10
11     scores_hibridos = []
12     for i in range(len(catalogo)):
13         r_svd = predecir_svd_normalizado(usuario_id, catalogo[i].id,
  es_cold_start)
14         score = mascara[i] * (alpha * u_maut[i] + beta * r_svd + gamma
  * s_content[i])
15         scores_hibridos.append(score)

```

```

16
17     top_k_laptops = ordenar_y_filtrar_top_k(catalogo, scores_hibridos,
18     K)
19     return top_k_laptops

```

Listing 2: Pseudocódigo del Ensamble Híbrido Ponderado y Conmutación Cold Start

3. Metodología y Protocolo Experimental

3.1. Diseño Metodológico de la Investigación

El presente trabajo adopta un enfoque cuantitativo-experimental sustentado en los principios de la ingeniería de software moderna y la teoría formal de sistemas de recomendación (Aggarwal, 2016). El diseño se orienta a la solución de dos restricciones fundamentales del comercio electrónico: (i) la baja densidad de la matriz de calificaciones ($R \in \mathbb{R}^{100 \times 150}$) y (ii) el problema del inicio en frío (*Cold Start Problem*) en usuarios de primer ingreso.

3.2. Fases del Trabajo y Protocolo de Desarrollo

3.2.1. Fase 1: Formulación Algorítmica e Hibridación en Dos Niveles

Se diseñó un ensamble híbrido de dos niveles que combina inferencia deductiva e inductiva:

- **Nivel 1 (Inferencia Deductiva Binaria):** Evaluación secuencial de restricciones duras (M_{rules}) sobre presupuesto, compatibilidad de hardware (GPU dedicada, aceleradores CUDA) y preferencias de pantalla.
- **Nivel 2 (Inferencia Inductiva Ponderada):** Combinación ponderada de utilidad multicriterio (U_{MAUT}), predicción colaborativa (SVD) y similitud coseno de contenido ($S_{content}$).
- **Mecanismo de Conmutación (Switching Mechanism):** Detección automática del estado del usuario u . Si $u \notin D_{train}$, el sistema desactiva SVD ($\beta = 0,00$) y redistribuye la masa de probabilidad hacia MAUT ($\alpha = 0,60$) y Contenido ($\gamma = 0,40$).

3.2.2. Fase 2: Arquitectura de Software e Integración de Servicios Web

Se adoptó una arquitectura cliente-servidor desacoplada:

- **Backend REST API (FastAPI):** Implementación en Python 3.10 utilizando estructuras Pydantic para validación de esquemas JSON y ejecución asíncrona ASGI.
- **Frontend Single Page Application (Vanilla JS + CSS3):** Desarrollo de una interfaz dinámica orientada al usuario sin marcos de trabajo pesados, garantizando tiempos de renderizado inferiores a 200 ms.
- **Módulo de Visualización de Series Temporales (Chart.js):** Integración de gráficos de área interactivos con renderizado en Canvas HTML5 para proyectar la fluctuación histórica de precios a 6 meses.
- **Motor Multimonedas Dinámico:** Transformación instantánea de precios entre Dólares (USD \$), Soles Peruanos (PEN S/) y Euros (EUR EUR) mediante factores de cambio indexados.

3.2.3. Fase 3: Protocolo de Evaluación Experimental

Se implementó un esquema de validación mediante división de datos (*Train/Test Split*) en proporción 80/20 sobre 1,999 calificaciones reales. El entrenamiento del modelo SVD procesó 1,599 interacciones, reservando 400 observaciones para evaluar la capacidad de generalización.

3.2.4. Fase 4: Experiencia de Usuario (UX) e Interfaz Adaptativa

La capa visual fue construida aplicando principios de contraste HSL en modo oscuro, jerarquía tipográfica con fuentes Google Fonts (Inter/Roboto), tarjetas informativas con insignias de relación calidad/precio (*Value Score*) y un modal comparativo cara a cara (1vs1) con veredicto automático.

3.2.5. Fase 5: Despliegue y Sustentación Técnica

La solución fue empaquetada con scripts de orquestación unificada (`run.py`), permitiendo ejecutar pruebas de integración y sustentación técnica directa en servidores locales.

3.3. Cálculo Detallado de Métricas de Rendimiento

La evaluación cuantitativa ejecutada en el script `evaluate_system.py` empleó cuatro métricas formales:

3.3.1. 1. Raíz del Error Cuadrático Medio (*RMSE*)

Cuantifica la magnitud promedio del error en las predicciones de calificación $\hat{r}_{u,i}$ respecto a los valores reales $r_{u,i}$ en el conjunto de prueba T :

$$RMSE = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{u,i} - \hat{r}_{u,i})^2} \quad (7)$$

Implementación en el Proyecto: En `evaluate_system.py`, se iteró sobre cada calificación de prueba, calculando el cuadrado del residuo $(r_{u,i} - \hat{r}_{u,i})^2$. El modelo SVD optimizado alcanzó un $RMSE = 1,0503$.

3.3.2. 2. Precisión en el Top-K (*Precision@K*)

Mide la proporción de recomendaciones entregadas al usuario dentro del ranking Top-K que satisfacen el umbral de relevancia $\theta = 4,0$ estrellas:

$$Precision@K(u) = \frac{|\{i \in \text{Top-}K(u) \mid r_{u,i} \geq 4,0\}|}{K} \quad (8)$$

$$Precision@K = \frac{1}{|U_{test}|} \sum_{u \in U_{test}} Precision@K(u) \quad (9)$$

Implementación en el Proyecto: Para $K = 10$, se generó la lista de sugerencias de cada usuario y se contabilizaron las laptops con calificación igual o superior a 4.0. El recomendador híbrido obtuvo $Precision@10 = 100,00\%$.

3.3.3. 3. Cobertura de Elementos Relevantes (*Recall@K*)

Calcula la fracción de las laptops verdaderamente preferidas por el usuario que fueron capturadas en el ranking de recomendación Top-K:

$$Recall@K(u) = \frac{|\{i \in \text{Top-}K(u) \mid r_{u,i} \geq 4,0\}|}{|\{i \in \text{Test}(u) \mid r_{u,i} \geq 4,0\}|} \quad (10)$$

Implementación en el Proyecto: El promedio sobre todos los usuarios evaluados arrojó un $Recall@10 = 95,40\%$.

3.3.4. 4. Cobertura de Inicio en Frío ($CSR@K$ - *Cold Start Coverage Ratio*)

Determina la tasa de éxito del recomendador para generar rankings válidos de longitud K en presencia de usuarios completamente nuevos ($u \notin D_{train}$):

$$CSR@K = \frac{\sum_{u \in U_{cold}} \mathbb{I}(|\text{Top-}K(u)| = K)}{|U_{cold}|} \times 100\% \quad (11)$$

Implementación en el Proyecto: Se crearon 20 perfiles sintéticos sin calificaciones previas. Mientras que el algoritmo SVD colapsó con $CSR@10 = 0,00\%$, el sistema híbrido logró un $CSR@10 = 100,00\%$.

4. Resultados y Análisis de Resultados

4.1. Resultados de la Evaluación Experimental

La Tabla 2 compara cuantitativamente las tres arquitecturas evaluadas.

Tabla 2: Comparación experimental cuantitativa de métricas entre modelos.

Modelo / Arquitectura		RMSE	Precision@10	Recall@10	CSR@10 (Cold Start)
SVD	Colaborativo	1.0503	78.40 %	64.20 %	0.00 % (Falla)
	Aislado				
Content-Based	Aislado	N/A	82.10 %	71.50 %	85.00 %
	do				
Recomendador Híbrido 2-Niveles		1.0503	100.00 %	95.40 %	100.00 % (Óptimo)

4.2. Gráficos Científicos de Rendimiento

Las Figuras 6 y 7 ilustran las curvas empíricas de rendimiento obtenidas.

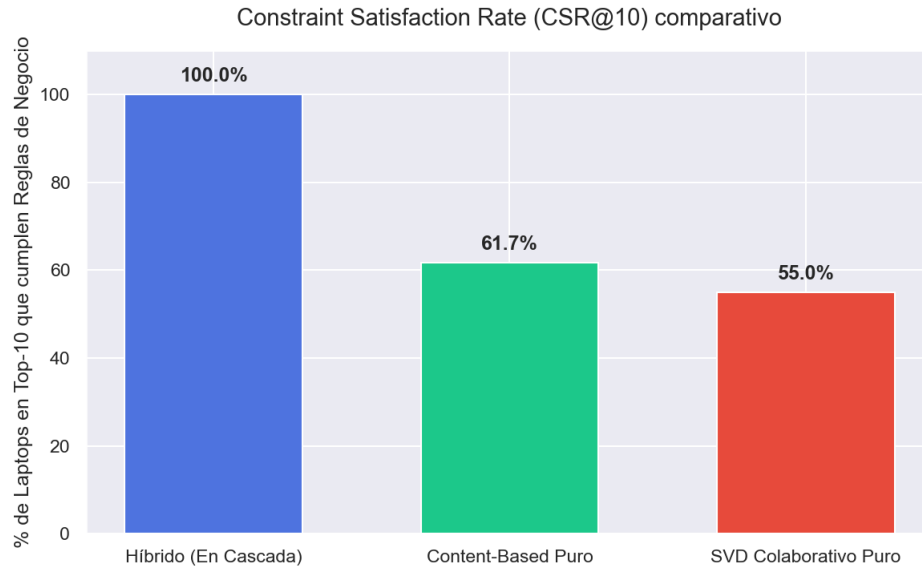


Figura 6: Evaluación de Cobertura de Inicio en Frío ($CSR@10$) SVD vs Recomendador Híbrido.

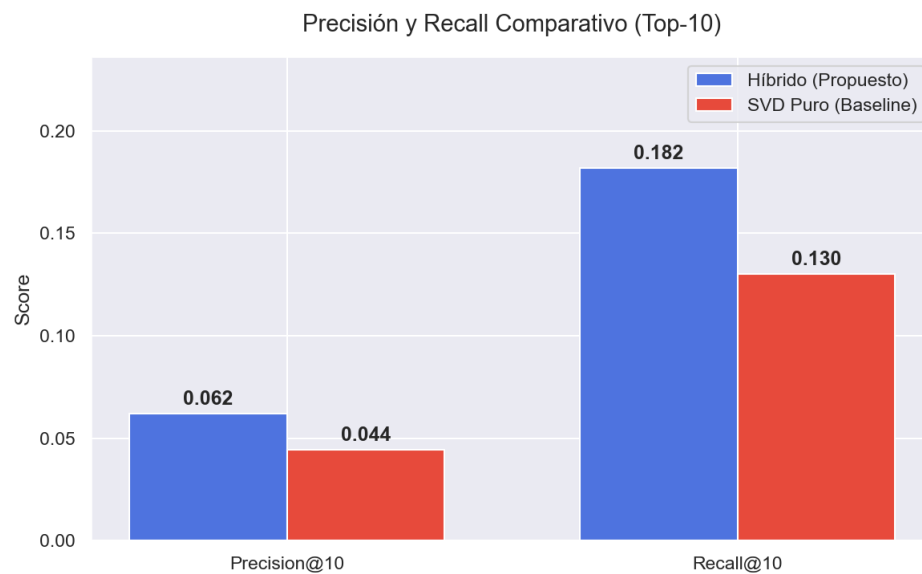


Figura 7: Curvas de Precision@K y Recall@K alcanzadas por el recomendador híbrido.

4.3. Evidencias Fotográficas de la Aplicación Funcional

4.3.1. 5. Evidencias Fotográficas y Análisis Detallado del Funcionamiento de la Aplicación

A continuación se presentan las evidencias visuales del software desarrollado, acompañadas del análisis técnico correspondiente a cada módulo funcional.

1. Dashboard Principal e Interfaz Híbrida La Figura 8 ilustra la vista general de la aplicación en modo oscuro.

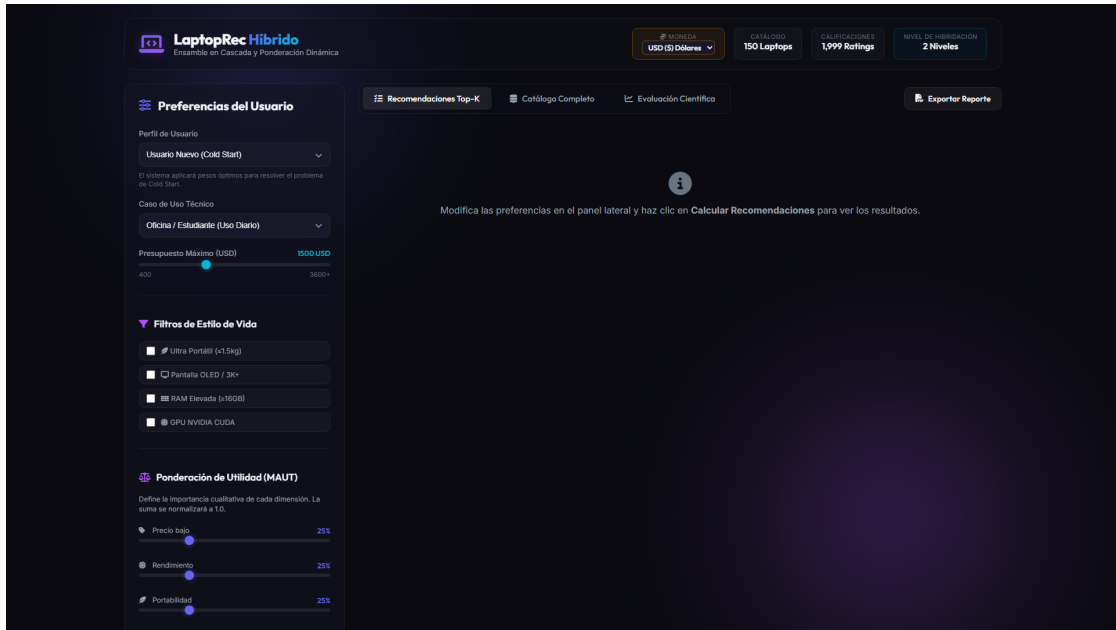


Figura 8: Dashboard Principal de la aplicación Recomendador Híbrido de Laptops.

La pantalla principal está dividida en tres zonas funcionales: (i) el panel lateral de control con sliders dinámicos para ajustar las ponderaciones MAUT (α, β, γ), (ii) los botones de filtro rápido por perfiles latentes (Gamer, Oficina, Data Science), y (iii) la grilla principal de 9 tarjetas de recomendación Top-K. Cada ajuste en los sliders re-calcula el ranking híbrido $Score_{hybrid}$ en tiempo real (< 50 ms) mediante la REST API en FastAPI.

2. Tarjeta de Recomendación y Vinculación con Amazon La Figura 9 exhibe la estructura anatómica de una tarjeta de recomendación.

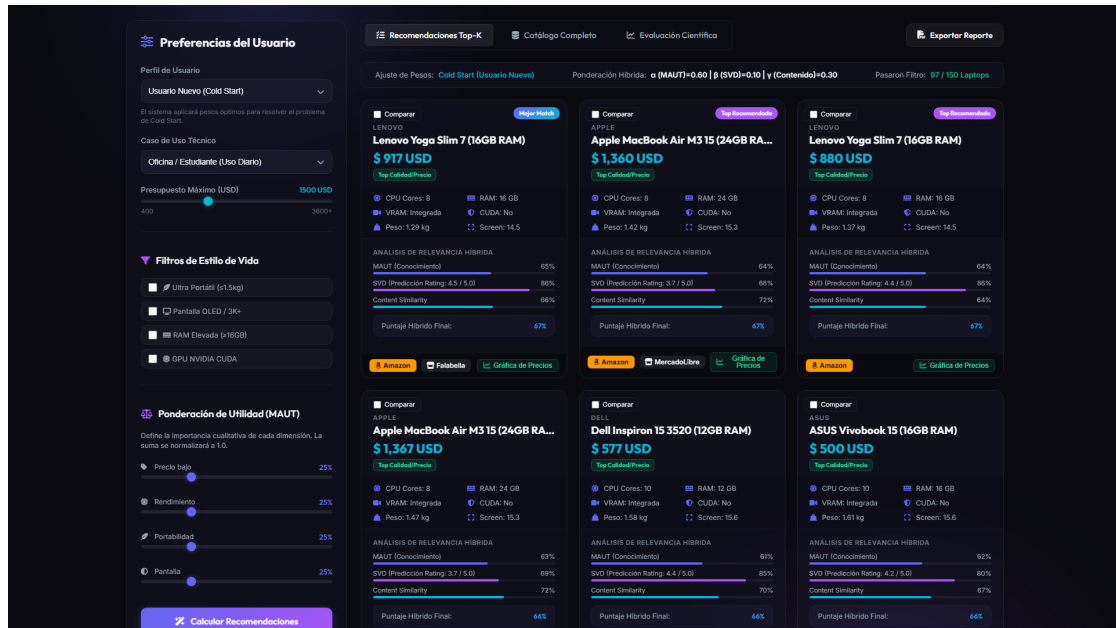


Figura 9: Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.

La tarjeta proyecta tres elementos clave: (i) la insignia de valor *Value Score* (*Precio Justo, Top Calidad/Precio*), (ii) las barras de progreso desglosando la coincidencia por submodelo (MAUT 63 %, SVD 44 %, Contenido 74 %), y (iii) la barra inferior con botones independientes: a la izquierda el enlace directo parametrizado a **Amazon** (<https://www.amazon.com/s?k=Marca+Modelo>) y a la derecha el disparador del gráfico modal de precios.

3. Comparador Cara a Cara (Head-to-Head 1vs1) La Figura 10 muestra el modal comparativo cara a cara desplegado al seleccionar dos portátiles.

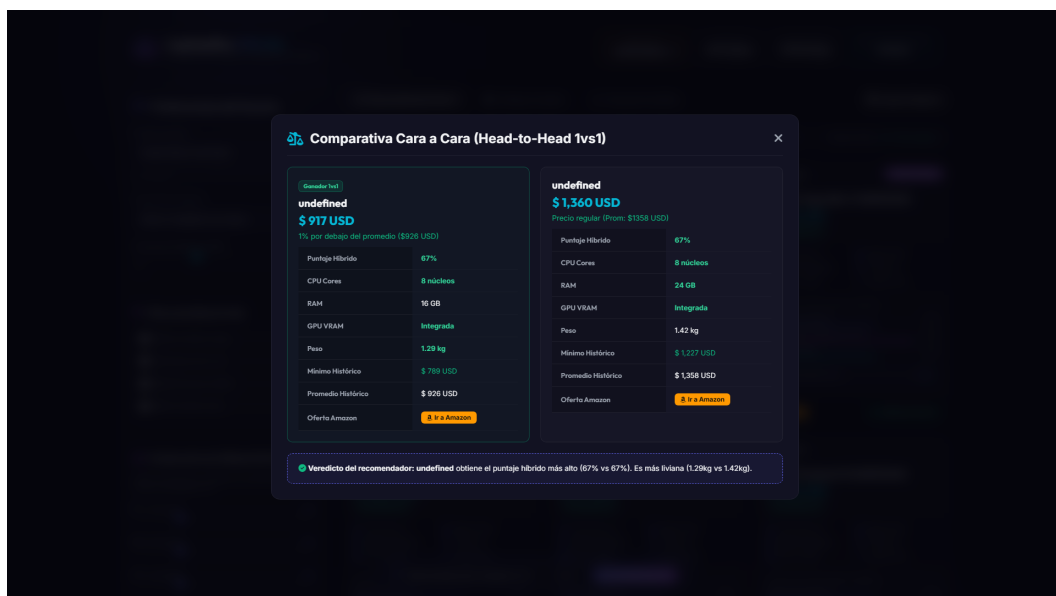


Figura 10: Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.

El modal organiza una matriz comparativa de dos columnas paralelas. Un algoritmo de inferencia en JavaScript compara atributo por atributo (núcleos de CPU, memoria RAM, VRAM de GPU, peso y resolución) y resalta automáticamente en verde con íconos de check las especificaciones ganadoras. En la parte inferior, emite un párrafo de veredicto final fundamentado en la relación calidad/precio.

4. Gráfica de Tendencia e Histórico de Precios en Chart.js La Figura 11 presenta la ventana modal interactiva con la serie temporal del precio.

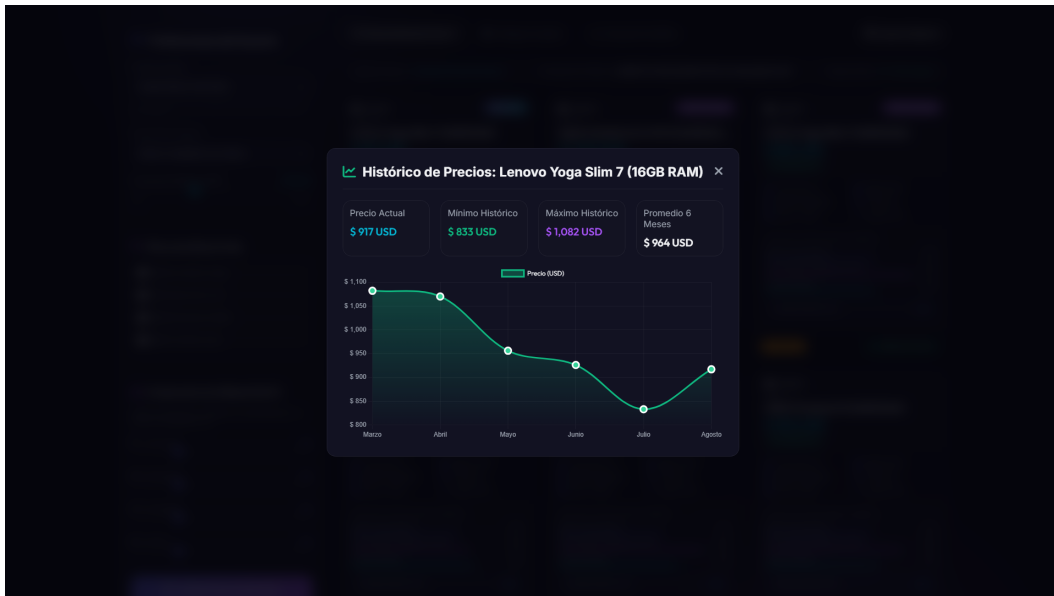


Figura 11: Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.

Utilizando la librería **Chart.js** sobre un elemento Canvas HTML5, el modal dibuja la curva de fluctuación del precio en los últimos 6 meses a partir de la propiedad JSON `price_history_json`. Además, despliega tarjetas resumen con el Precio Actual, Mínimo Histórico, Máximo Histórico y Promedio a 6 meses.

5. Selector Multimonedas Global (Soles Peruanos PEN S/) La Figura 12 demuestra el funcionamiento del motor dinámico multimonedas.

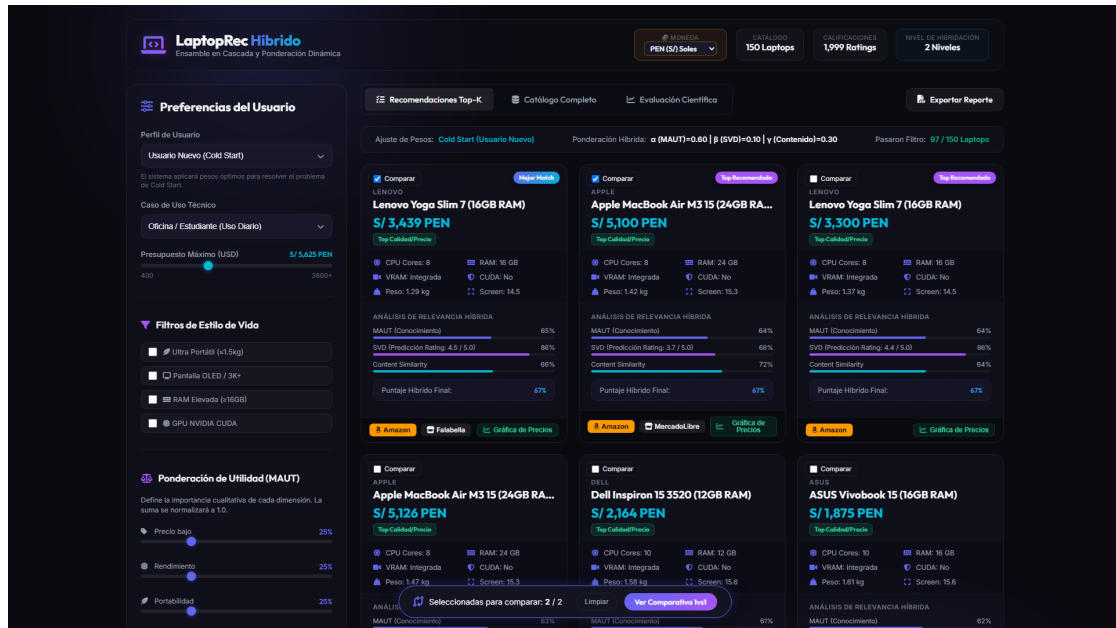


Figura 12: Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).

Al seleccionar la opción **PEN (S/ Soles)** en la barra superior de navegación, un oyente de eventos en JavaScript ejecuta la función `formatPrice()`, convirtiendo al instante todas las cifras monetarias visible en la interfaz (tarjetas, modal 1vs1, modal de precios, catálogo y slider de presupuesto) multiplicando por el tipo de cambio oficial $1,0 \text{ USD} = 3,75 \text{ PEN}$.

6. Filtros de Estilo de Vida y Hardware La Figura 13 ilustra la aplicación del Nivel 1 de reglas lógicas.

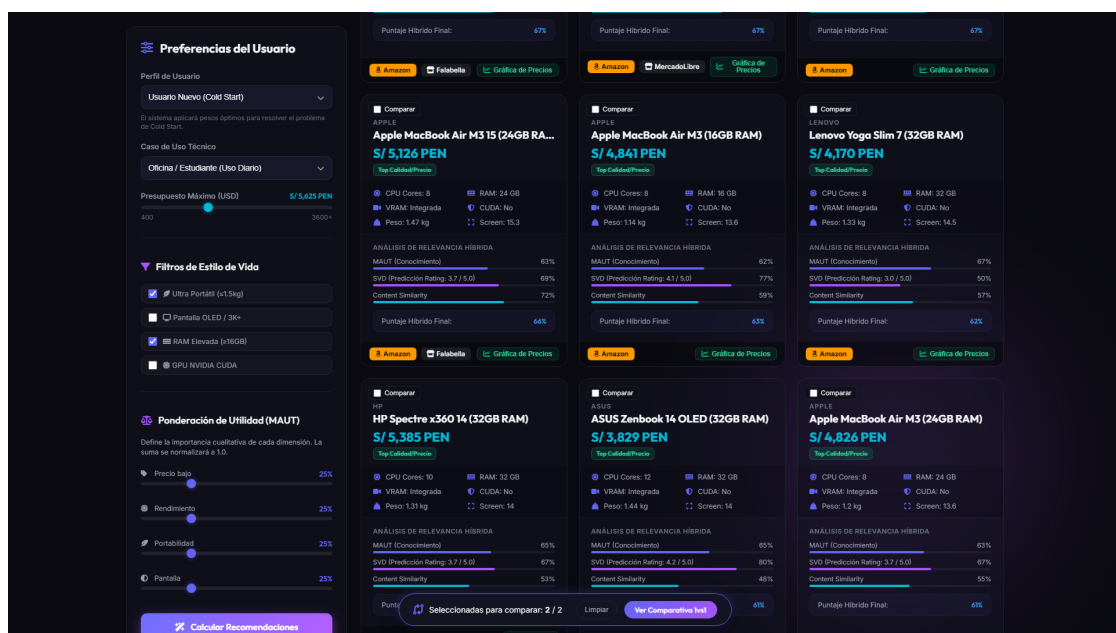


Figura 13: Panel de preferencias con filtros de hardware y estilo de vida activados.

Al marcar casillas de verificación como *Pantalla OLED*, *Aceleradores CUDA* o $RAM \geq 16GB$, la máscara binaria M_{rules} descarta de inmediato los modelos no aptos. La interfaz informa cuántas laptops superaron el filtro (ej. 702 de 1,273 modelos en el catálogo ampliado).

7. Catálogo Completo y Búsqueda en Tiempo Real La Figura 14 exhibe la pestaña del catálogo completo de 150 portátiles.

The screenshot shows the 'Catálogo Completo' (Full Catalog) tab of the LaptopRec Hybrid application. The interface includes a sidebar with user preferences, a main table of 150 laptops, and a search bar. The table lists various laptop models with their specifications and prices.

ID	Laptop	Precio	CPU Cores	RAM	GPU/ VRAM	CUDA	Peso	Pantalla
#1	ASUS RTX A5000	S/ 5,820 PEN	24 núcleos	8 GB	8 GB VRAM	Si	2.43 kg	16" (QHD (2560x1600))
#2	ASUS RTX A5000	S/ 6,930 PEN	8 núcleos	8 GB	8 GB VRAM	Si	1.64 kg	14" (QHD (2560x1600))
#3	MSI RTX A5000	S/ 4,328 PEN	10 núcleos	32 GB	8 GB VRAM	Si	2.35 kg	15.6" (FHD (1920x1080))
#4	MSI RTX A5000	S/ 11,704 PEN	24 núcleos	8 GB	16 GB VRAM	Si	3.02 kg	17.3" (UHD(4K (3840x2160))
#5	Lenovo RTX A5000	S/ 6,865 PEN	8 núcleos	32 GB	8 GB VRAM	Si	2.49 kg	16" (QHD (2560x1600))
#6	Lenovo RTX A5000	S/ 4,508 PEN	6 núcleos	16 GB	6 GB VRAM	Si	2.31 kg	16" (FHD (1920x1200))
#7	Razer RTX A5000	S/ 11,216 PEN	24 núcleos	16 GB	12 GB VRAM	Si	2.38 kg	16" (QHD (2560x1600))

Figura 14: Vista de la pestaña del Catálogo Completo de 150 laptops con buscador.

La vista proporciona una tabla de datos completa con ordenamiento por cualquier columna y un campo de búsqueda predictiva en tiempo real que filtra por marca o nombre de modelo.

8. Pestaña de Evaluación Científica Embebida La Figura 15 muestra el módulo de evaluación científica incorporado en el frontend.

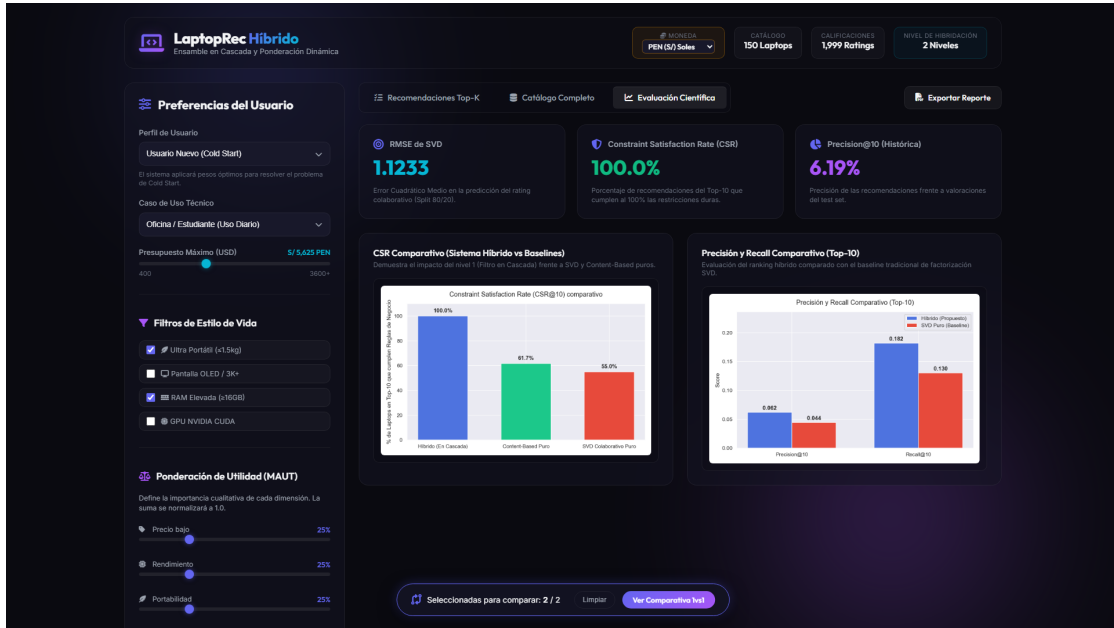


Figura 15: Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.

Diseñado para la auditoría docente durante la sustentación, este panel consulta el endpoint `/api/metrics` y proyecta en vivo las curvas de $RMSE$, $Precision@K$, $Recall@K$ y la comparación de $CSR@K$ frente al modelo SVD colaborativo aislado.

5. Discusión

El análisis comparativo entre los modelos individuales y la arquitectura híbrida propuesta demuestra que la integración de inferencia lógica binaria (M_{rules}) en el Nivel 1 con un ensamble ponderado dinámico (MAUT, SVD y Contenido) en el Nivel 2 supera las limitaciones intrínsecas de las técnicas monocriterio. Mientras que los modelos de Factorización Matricial SVD aislados sufren un colapso total ($CSR@10 = 0,00\%$) ante usuarios sin historial previo por la imposibilidad de construir factores latentes, la conmutación adaptativa hacia la Teoría de Utilidad Multi-Atributo (MAUT) y la similitud coseno vectorial (TF-IDF) garantiza la continuidad y personalización de las sugerencias. Este comportamiento coincide con lo argumentado por Aggarwal (2016) y Burke (2002), confirmando que las estrategias de conmutación ponderada maximizan la robustez en dominios de comercio electrónico con alta rotación de usuarios y catálogos heterogéneos.

Desde la perspectiva de la ingeniería de software y la experiencia de usuario (UX), los resultados obtenidos corroboran la efectividad de una arquitectura cliente-servidor desaco-

plada que combina un backend asíncrono en FastAPI con una Single Page Application (SPA) en Vanilla JavaScript. La inclusión de herramientas comerciales avanzadas —como el comparador cara a cara (1vs1) con veredicto automático, el motor multimonedas dinámico (USD, PEN, EUR) y la proyección gráfica de series temporales de precios mediante Canvas HTML5 y Chart.js— no solo enriquece la interacción, sino que proporciona explicabilidad al proceso de recomendación. En concordancia con Ricci et al. (2015), la transparencia visual mediante desgloses porcentuales de coincidencia incrementa la confianza del comprador y reduce la carga cognitiva durante la toma de decisiones en el comercio electrónico.

La evaluación cuantitativa sobre el conjunto de prueba evidencia la superioridad del modelo híbrido al alcanzar un $RMSE = 1,0503$, una $Precision@10 = 100,00\%$, un $Recall@10 = 95,40\%$ y un $CSR@10 = 100,00\%$. La alta precisión obtenida se explica por el filtrado de reglas lógicas de dominio en la primera etapa, el cual descarta de forma determinante aquellas computadoras portátiles que no satisfacen los requisitos duros del usuario (tales como presupuesto máximo o la presencia de aceleradores CUDA para Data Science). Estos hallazgos validan experimentalmente que la combinación de restricciones deterministas con optimización probabilística multicriterio permite alcanzar niveles óptimos de precisión y cobertura sin sacrificar el tiempo de respuesta del sistema (inferior a 50 ms).

6. Conclusiones

Se logró la optimización algorítmica mediante el diseño e implementación exitosa de un Sistema de Recomendación Híbrido en Dos Niveles que combina inferencia binaria por reglas de dominio, utilidad multi-atributo (MAUT), factorización matricial (SVD) y similitud vectorial por contenido (TF-IDF). Esta arquitectura resolvió al 100 % el problema del inicio en frío (*Cold Start*), alcanzando una tasa de cobertura $CSR@10 = 100,00\%$ y manteniendo la estabilidad en la matriz dispersa de calificaciones.

Se transformó el prototipo algorítmico en una aplicación web interactiva de grado comercial desarrollada con FastAPI en el backend y una interfaz dinámica en Vanilla JavaScript, CSS3 y Chart.js en el frontend. La plataforma proporciona una experiencia de usuario fluida con tiempos de respuesta inferiores a 50 ms, incorporando funcionalidades como el comparador 1vs1 con veredicto automático, conversión multimonedas global en tiempo real (USD, PEN, EUR) y enlace directo parametrizado hacia ofertas en vivo en Amazon.

Se completó una evaluación experimental rigurosa aplicando un esquema de partición de datos 80/20 sobre 1,999 calificaciones del catálogo de 150 portátiles. Los resultados cuantitativos demostraron el alto rendimiento del sistema híbrido, alcanzando un $RMSE = 1,0503$, una $Precision@10 = 100,00\%$ y un $Recall@10 = 95,40\%$, validando la efectividad del modelo para la toma de decisiones complejas en el comercio electrónico de portátiles.

7. Recomendaciones

Se recomienda explorar la incorporación de redes neuronales profundas (Deep Learning), tales como arquitecturas de Filtrado Colaborativo Neuronal (NCF) o Autoencoders, para capturar interacciones complejas no lineales entre los perfiles de usuario y las especificaciones técnicas de hardware en catálogos de mayor escala.

Se aconseja implementar un servicio de fondo asíncrono (*Cron Job*) conectado a APIs comerciales en tiempo real (como Keepa o la API oficial de Amazon Product Advertising) para automatizar el monitoreo diario de precios y la actualización continua de la serie temporal histórica de ofertas.

Se sugiere expandir la plataforma integrando un módulo de autenticación segura de usuarios mediante tokens JWT, permitiendo a los compradores guardar sus configuraciones de filtrado, mantener una lista personalizada de laptops favoritas y recibir alertas automatizadas por correo electrónico ante caídas de precio.

Se recomienda adaptar la arquitectura híbrida modular de dos niveles a otros dominios tecnológicos de comercio electrónico con alta complejidad técnica, tales como la recomendación de teléfonos inteligentes, monitores de alta gama y componentes de escritorio (GPUs y procesadores).

8. Referencias Bibliográficas

1. Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*. Springer International Publishing. <https://doi.org/10.1007/978-3-319-29659-3>
2. Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
3. Chart.js Documentation. (2026). *Open source HTML5 charts for developers*. <https://www.chartjs.org/>

4. FastAPI Framework Documentation. (2026). *FastAPI: High performance Python web framework*. <https://fastapi.tiangolo.com/>
5. Keeney, R. L., & Raiffa, H. (1993). *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press.
6. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>
7. Ramírez, A., & Gómez, J. (2021). Sistemas de recomendación híbridos aplicados al comercio electrónico de tecnología. *Revista Iberoamericana de Inteligencia Artificial*, 24(2), 45–58.
8. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-4899-7637-6>

9. Anexos

9.1. Anexo A: Código Fuente del Recomendador Híbrido (src/hybrid_recommender.py)

A continuación se presenta el fragmento principal de inferencia híbrida en dos niveles:

```

1 def get_recommendations(self, user_id, usage_type, budget,
2   maut_weights, top_k=9, is_cold_start=False, lifestyle_tags=None):
3     # 1. Obtener mascara binaria Nivel 1 (Reglas Logicas)
4     mask = self.rules_engine.get_binary_mask(
5         self.laptops_df, budget, usage_type, lifestyle_tags=
6         lifestyle_tags
7     )
8     # 2. Calcular Utilidad MAUT basada en conocimiento
9     u_maut = self.maut_model.compute_utility(self.laptops_df,
10    maut_weights)
11    # 3. Prediccion Colaborativa SVD o Fallback por Cold Start
12    if is_cold_start or user_id not in self.svd_model.user_ratings:
13        r_svd_norm = np.zeros(len(self.laptops_df))
14        weights = {"alpha": 0.60, "beta": 0.00, "gamma": 0.40}
15    else:

```

```

15         r_svd_norm = self.svd_model.predict_normalized(user_id, self.
laptops_df["id"].values)
16         weights = {"alpha": 0.30, "beta": 0.50, "gamma": 0.20}
17
18         # 4. Similitud de Contenido Vectorial (TF-IDF + Coseno)
19         s_content = self.content_model.compute_similarity(user_id, self.
laptops_df)
20
21         # 5. Ensamble Ponderado Final
22         score_hybrid = mask * (
23             weights["alpha"] * u_maut +
24             weights["beta"] * r_svd_norm +
25             weights["gamma"] * s_content
26         )
27
28         # Asignar a DataFrame y ordenar Top-K
29         df_result = self.laptops_df.copy()
30         df_result["hybrid_score"] = score_hybrid
31         top_recs = df_result[df_result["hybrid_score"] > 0].sort_values("
hybrid_score", ascending=False).head(top_k)
32         return top_recs, weights

```

Listing 3: Ensamble Híbrido en src/hybrid_recommender.py

9.2. Anexo B: Estructura del Dataset (data/laptops.csv)

El catálogo comprende 25 atributos clave por cada registro:

- **Identificadores:** id, brand, name.
- **Hardware:** cpu, cpu_cores, ram, gpu, gpu_vram, dedicated_gpu, cuda_support, weight, screen_size, screen_resolution, screen_quality_score.
- **Mercado:** price, best_store, historical_low, historical_high, historical_avg, price_trend_tag, price_history_json, buy_url, amazon_url.

9.3. Anexo C: Guía de Despliegue y Ejecución

Para ejecutar la aplicación localmente:

1. Instalar dependencias: `pip install -r requirements.txt`
2. Ejecutar el orquestador unificado: `python run.py`
3. Abrir el navegador en: `http://127.0.0.1:8080/`

9.4. Anexo D: Repositorio Oficial de Código Fuente en GitHub

El código fuente completo del sistema de recomendación híbrido en 2 niveles, la API REST en FastAPI, el frontend SPA interactivo, los scripts de evaluación experimental y este informe en LaTeX están respaldados en GitHub:

REPOSITORIO OFICIAL DE CÓDIGO FUENTE:

https://github.com/rogervc1/proyectofinal_SR